Topic 10 | Operational Lead Tracker

Workflow reference converted from the supplied n8n JSON. This PDF is for reading and documentation; use the original JSON for n8n import.

Workflow overview

#	Node	Purpose / configuration
1	Manual Trigger	manualTrigger
2	Read Operational Leads	googleSheets | read
3	Filter Pending Leads	code | JavaScript transformation
4	Process Leads One by One	splitInBatches
5	Prepare Processing Lock	code | JavaScript transformation
6	Lock Row - Set Processing	googleSheets | update
7	Simulate Business HTTP Action	httpRequest | POST https://httpbin.org/post
8	Evaluate Business Result	code | JavaScript transformation
9	IF Business Action Succeeded	if
10	Update Row - Completed	googleSheets | update
11	Update Row - Failed	googleSheets | update
12	Audit Result	code | JavaScript transformation

Execution flow

Manual Trigger -> Read Operational Leads

Read Operational Leads -> Filter Pending Leads

Filter Pending Leads -> Process Leads One by One

Process Leads One by One -> Prepare Processing Lock [output 1]

Prepare Processing Lock -> Lock Row - Set Processing

Lock Row - Set Processing -> Simulate Business HTTP Action

Simulate Business HTTP Action -> Evaluate Business Result

Evaluate Business Result -> IF Business Action Succeeded

Update Row - Completed -> Audit Result

Update Row - Failed -> Audit Result

Audit Result -> Process Leads One by One

IF Business Action Succeeded -> Update Row - Completed [output 0]

IF Business Action Succeeded -> Update Row - Failed [output 1]

Configuration notes

The source JSON includes Google Sheets document references, credential references, and one placeholder spreadsheet ID. Check all Google Sheets nodes and reconnect credentials in your own n8n instance before running.

Node configuration details

Manual Trigger

Read Operational Leads

documentId: 1YlNZXEPTN36hwjjp8pU6mweRpfyxDiLGaOk8oCV9EOk

sheetName: gid=0

Filter Pending Leads

JavaScript: // Empty Google Sheets reads may produce a single empty item (Always Output Data).
// Only forward real Pending records with a nonempty lead_id.
const items = $input.all();
return items.filter(item => {
const r = item.json || {};
return String(r.lead_status ?? '').trim().toLowerCase() === 'pending'
&& String(r.lead_id ?? '').trim().length > 0;
}).map(item => ({json:{...item.json,lead_id:String(item.json.lead_id).trim()}}));

Process Leads One by One

Prepare Processing Lock

JavaScript: const lead=$input.first().json; return
[{json:{...lead,lead_status:'Processing',execution_id:String($execution.id||'manual-test'),updated_at:new
Date().toISOString(),error_details:''}}];

Lock Row - Set Processing

operation: update

documentId: 1YlNZXEPTN36hwjjp8pU6mweRpfyxDiLGaOk8oCV9EOk

sheetName: gid=0

Columns: matching=['error_details']; mappings={}

Simulate Business HTTP Action

method: POST

url: https://httpbin.org/post

sendBody: True

specifyBody: json

jsonBody: ={{ JSON.stringify({lead_id: $('Prepare Processing Lock').first().json.lead_id, name: $('Prepare Processing Lock').first().json.name, email: $('Prepare Processing Lock').first().json.email}) }}

Evaluate Business Result

JavaScript: const locked=$('Prepare Processing Lock').first().json;const r=$input.first().json||{};const
status=Number(r.statusCode||r.error?.httpCode||0);const ok=!r.error&&status>=200&&status<300;return
[{json:{...locked,business_status_code:status,action_ok:ok,lead_status:ok?'Completed':'Failed',updated_at:new

Date().toISOString(),error_details:ok?'':String(r.error?.message||r.message||('HTTP status '+status)).slice(0,500)}}];

IF Business Action Succeeded

Conditions: {"options": {"caseSensitive": true, "leftValue": "", "typeValidation": "strict", "version": 1}, "conditions": [{"id": "cf3449a8-a04f-4e61-9f24-b1686798c827", "leftValue": "={{ $json.action_ok }}", "rightValue": true, "operator": {"type": "boolean", "operation": "true", "singleValue": true}}], "combinator": "and"}

Update Row - Completed

operation: update

documentId: 1YINZXEPTN36hwjjp8pU6mweRpfyxDiLGaOk8oCV9EOk

sheetName: gid=0

Columns: matching=['lead_id']; mappings={"lead_id": "={{ $json.lead_id }}", "name": "={{ $json.name }}", "email": "={{ $json.email }}", "lead_status": "={{ $json.lead_status }}", "execution_id": "={{ $json.execution_id }}", "updated_at": "={{ $json.updated_at }}", "error_details": "={{ $json.error_details }}"}

Update Row - Failed

operation: update

documentId: REPLACE_WITH_SPREADSHEET_ID

sheetName: Operational_Lead_Tracker

Columns: matching=['lead_id']; mappings={"lead_id": "={{ $json.lead_id }}", "lead_status": "={{ $json.lead_status }}", "execution_id": "={{ $json.execution_id }}", "updated_at": "={{ $json.updated_at }}", "error_details": "={{ $json.error_details }}"}

Audit Result

JavaScript: const evaluated = $('Evaluate Business Result').first().json;
const sheetResponse = $input.first().json || {};
return [{json:{lead_id:evaluated.lead_id,lead_status:evaluated.lead_status,execution_id:evaluated.execution_id,updated_at:evaluated.updated_at,error_details:evaluated.error_details||'',sheetUpdateReturnedError:!!sheetResponse.error,audit_message:'Row update attempted; check the Google Sheet to verify persistence.'}}];